

Oct 15, 2025

Meeting Oct 15, 2025 at 15:17 CEST

Meeting records  Recording

Summary

Clément Lacaille led a session where Naim Burrack volunteered to demonstrate connecting to the SER API for job searches. Naim encountered challenges with SER API account creation and phone number verification, ultimately resolving them to successfully retrieve an API key and configure the HTTP request node. The discussion also involved troubleshooting search queries, refining AI system messages for precise results, and using a custom code node in N8N to process API output, with contributions from Kai Feldhoff, Sven Lück, Leon Winde, Julius-Jonas Steffen, and Alex Matviiets.

Details

Notes Length: Standard

- **Workflow Completion and Volunteer Request** Clément Lacaille observed that several participants, including Naim Burrack, had completed the initial steps of the workflow. Clément requested a volunteer to share their screen and demonstrate the subsequent steps for the day, offering guidance throughout the process. Naim volunteered to continue if no one else wished to, and Clément agreed to let Naim proceed due to a lack of other volunteers.
- **Resuming the Workflow and Initial Steps** Naim shared their screen and confirmed they had the previous day's chat window. Naim mentioned they had repeated the initial steps during a lunch break after deleting their own work. Clément instructed Naim to take a screenshot of the current status on Netten, post it on publicity, and copy and paste the JSON output from Open AI into the

chat, then start writing about the goal of connecting to SER API to search for jobs.

- **Connecting to SER API and User Prompt** Naim expressed their intent to connect to SER API to search for jobs that fit their search request. Naim then copied and pasted system and user messages for the SER API from an earlier example for everyone to use. Naim confirmed that the output from publicity indicated readiness to set up SER API and the next step was to create a SER API account and key.
- **SER API Account Creation and Verification Issues** Naim proceeded to create a SER API account, encountering a requirement to verify their email and phone number. Clément asked Naim to move to a quieter location due to background noise, and Naim confirmed they could address the issue. Naim attempted to sign in directly, but the site consistently directed them to a free plan requiring verification.
- **Phone Number Verification Challenges** Naim explored options for phone verification, considering using a fake number. Kai Feldhoff noted that a valid phone number was required for the code. Naim discovered many common numbers were already used. Clément offered their phone number for Naim to use. Sven Lück suggested going directly to the dashboard, but Naim found that a phone number was still required. Naim ultimately decided to use their own phone number.
- **Successful SER API Registration and API Key Retrieval** Naim successfully signed up for SER API, noting that all previously tried phone numbers had been used. Naim then shared their screen, showing the free plan with 250 searches. Clément instructed Naim to guide the class on how to obtain the API key. Naim explained how to copy the API key and paste it into the HTTP request node as a query parameter.
- **Preparing for SER API Request and Content Validation** Clément advised Naim to ask Perplexity if the Open AI response could be used directly with SER API or if additional steps were needed. Naim confirmed that the context was already present and asked if SER API could process the request directly. The response indicated that the content needed to be fixed to match the goal, as it was based on another user's file and not yet changed.
- **Configuring HTTP Request Node and Initial API Call** Naim configured the HTTP request node, setting the method to "GET" and inputting the URL. Clément

encouraged Naim to follow the instructions and take screenshots to ask for help if stuck, emphasizing the learning process. Naim agreed, noting the slow pace was beneficial for other classmates to follow along. Sven Lück suggested using the API key in the authentication field for safety, rather than as a query parameter.

- **Addressing API Key Security and Query Parameter Issues** Naim presented the issue of their JSON query not being recognized in the HTTP request node. Clément provided a solution by having Naim drag the "content" field from the left-hand side into the "Q" parameter's value field. Upon running the request, Naim received job results, although they contained some irrelevant data. Sven Lück clarified that the "Q" value contained the entire AI response, not just a job description, leading to less precise results.
- **Troubleshooting Search Queries and Location Bias** Clément suggested simplifying the search query to "backend developer" or "junior developer" within inverted quotes. After attempting several variations and encountering empty results, Clément and Naim discussed the possibility of language or location issues. Sven Lück suggested adding "google_domain" with value "google.de" and "HL" and "GL" parameters with value "DE" to target German Google results.
- **Collaborative Troubleshooting and Successful Job Search** After further attempts and continued empty results, Clément suggested changing the location to "New York" to isolate the problem. Leon Winde shared a working query string for Germany that included parentheses, which Naim then tried. Sven Lück shared their successful setup, including specific parameters like "web enticer" as the job title, "Google jobs" as the engine, and "google_domain: google.de," "HL: DE," and "GL: DE". Applying Sven's settings, Naim finally obtained relevant job listings.
- **Workflow Complexity and Future Steps** Clément acknowledged the complexity of Sven's workflow and expressed the intent to avoid the looping functions for beginners, praising Sven's accomplishment. Kai Feldhoff mentioned the N8N community documentation's AI agent as a helpful resource. Clément reminded participants to raise their hand before speaking to maintain order. Clément noted that the next step was to refine the Open AI content field to ensure precise location and job search queries for the SER API.
- **System Message Update and Search Query Constraints** Naim Burrack and Clément Lacaille discussed refining the system message for an AI model to improve search query generation. They focused on reducing the length of skills and location descriptions, ensuring only one search query is created to avoid

multiple API requests, and defining location as a separate parameter. Clément Lacaille emphasized that creating multiple search queries would lead to increased complexity and API costs.

- **API Token Usage and Page Refresh** Naim Burrack observed their API usage showing 12 searches. Sven Lück suggested refreshing the page because the API page was not auto-refreshing, noting that their own usage jumped from 12 to 50 after a refresh. Naim Burrack also asked attendees not to screenshot their code for security.
- **Refining Search Query Parameters** Naim Burrack and Clément Lacaille continued to refine the system message to include more restrictions, such as "no HR management terms" and "no extra text". Clément Lacaille clarified that location should be a separate parameter and the search query should combine job title and skill. They also discussed specifying the return to only include location and search query.
- **German Job Titles and Boolean Search Strings** Naim Burrack suggested including the German word for developer, "entwickler," in the search query. They also discussed building a boolean search string of 140 characters that includes the German job title and a compact skills group. Clément Lacaille confirmed this approach seemed effective.
- **Sharing and Customizing System/User Messages** Clément Lacaille requested Naim Burrack to share the updated system and user messages for others to copy and modify. Naim Burrack explained how users could customize the job title (e.g., from "developer" to "web marketing") and target country/city within the messages. They also suggested using AI to generate user messages that match a custom system message.
- **Switching to Claude for Code Generation** Clément Lacaille advised switching the AI model to Claude 4.5 for code generation, stating that Claude is better at writing code than OpenAI. Naim Burrack confirmed Perplexity app already included Claude. Clément Lacaille paused the session to allow others to update their system and user prompts and get output.
- **Using a Custom Code Node in N8N** Clément Lacaille guided Naim Burrack on using a custom code node in N8N to separate the search query and location into distinct columns from the OpenAI output. Sven Lück advised changing the code node mode from "all items" to "each item" for proper execution when using "first" inside the node. Naim Burrack resolved the issue of empty values by providing

the exact JSON input structure from OpenAI to the Claude model, which then generated the correct code.

- **Removing Brackets from Search Query Results** Naim Burrack and Clément Lacaille identified that the search query results contained unnecessary brackets. They iterated with the AI model, providing feedback that they did not want brackets, until the generated code successfully stripped them out, making the output more readable. Naim Burrack then shared the refined code in Slack for others.
- **Troubleshooting Code Node Issues** Clément Lacaille and Alex Matviiets troubleshooted an issue where Alex's code node returned empty values due to differences in the OpenAI output, specifically extra slashes. Clément Lacaille guided Alex to provide their exact JSON input from OpenAI to Claude, which then generated the corrected code. Clément Lacaille also offered to provide further assistance after the class for those still struggling with the code.
- **Breakout Rooms for Peer Support** Naim Burrack suggested using breakout rooms for peer support, which Clément Lacaille supported, to help classmates with issues. Clément Lacaille explained the limitations of breakout rooms in Google Meet but still created them, encouraging equal distribution of participants.
- **Dynamic Data Input in HTTP Request** Julius-Jonas Steffen raised a question about the workflow's reusability when input changes, specifically about manually entering job titles in the HTTP request. Clément Lacaille clarified that Julius-Jonas Steffen should drag and drop the "search query" schema into the value field of the HTTP request, enabling the system to automatically pick up the job title from the AI's output, making the workflow dynamic for different resumes.
- **Reflecting on Learning and Practice** Clément Lacaille encouraged participants to share one thing they learned from the session in the Slack group, highlighting the fun of coding with AI even without prior coding knowledge. They emphasized that Perplexity is an excellent teacher, advising participants to practice independently to master the process.

Suggested next steps

- ☐ Naim Burrack will take a screenshot of all current statuses on Net 10 and post it on publicity, including the output from the open AI JSON, then copy and paste the JSON.
- ☐ Naim Burrack will take a screenshot of the entire SER API window, focusing on the value entry section, and upload it to publicity.
- ☐ Naim Burrack will write in publicity to have open AI extract text from his resume to create a search query and location for SER API, addressing the current long output query from open AI.
- ☐ Clément Lacaille will check and get back to Naim Burrack about the form for last week that was required for the certificate.
- ☐ Naim Burrack will update his OpenAI messages to (1) include instructions to create only one search query, (2) mention that location and job title should be separate parameters, and (3) return only location (including a city) and search query (including the job title).

You should review Gemini's notes to make sure they're accurate. [Get tips and learn how Gemini takes notes](#)

Please provide feedback about using Gemini to take notes in a [short survey](#).